Customer handling intermediate server — an ‘architecture-led’ project

G Mathieson

The customer handling intermediate server (CHIS) project is seeking to pioneer the implementation of key elements of a three-tier client/server architecture for BT’s operational support systems (OSS).

CHIS makes comprehensive use of the Open Group’s Distributed Computing Environment (DCE).

The paper explains the justification for CHIS, outlines its scope and design principles, and discusses the practical difficulties, both technical and managerial, which have been experienced in its initial stages, and the solutions adopted.

1. Introduction

1.1 Customer handling intermediate server (CHIS) — history

BT’s major operational support systems (OSS) development of the 1980s was its integrated customer service system (CSS) which, alongside other major MVS-based systems for private circuit provision and fault repair and for 0800-type services, still forms the core of BT’s OSS.

These traditional mainframe systems are highly integrated and ‘mission critical’. The scale and range of BT’s operations, and the continual business demand for changes to support process improvements and automation, new product launches, and customer billing enhancements, result in complex change management and release processes, and the perception by business operational management of long lead times for development.

The increasing pace of change in BT’s business environment and organisation in the 1990s cannot be fully supported in this environment. Moreover, contemporary package solutions tend to have superior usability to the legacy systems, particularly for inexperienced users, though they typically cannot be scaled up to BT volumes or easily integrated with the rest of BT’s OSS.

A further perceived deficiency of the legacy applications is that they fail to support a customer-focused view of sales and service, particularly for larger accounts.

Part of the response to this problem has been a proliferation of ‘stove pipe’ solutions for the launch of new products, and the construction of overlay systems to address the needs of specific service channels. Additionally, a wide range of robotic PC-based applications have been developed, which ‘screen scrape’ the legacy systems to provide tactical solutions to process-automation requirements, and a number of overlay database systems have been created to provide a customer-focused view of data, with inevitable associated difficulties of data build, consistency and security.

Though valuable in the short term, such developments have been perceived as increasing the complexity of the OSS overall, increasing support costs, constructing further barriers to change and increasing the difficulty of providing sales and service support users with a comprehensive single view of BT’s relationship with its customers.

The strategic aims of any new architecture-led approach are, therefore, to support rapid launch of new products and services without expanding the OSS or adding unnecessary complexity to existing mainframe systems, thereby reducing costs.

These aims are clearly laudable in principle, but somewhat hard to achieve in practice.

In practice, the larger the user base, and the complexity and the absolute size of a system, the less amenable it is to change — hence the trend to broaden the range of discrete OSS systems. Reversing this trend can only be achieved by large-scale application of the principles of software engineering.

It is now considered impractical to replace legacy system functions *en masse* due to the uncertainties and long time-scales associated with very large system developments. Accordingly, although, as shall be seen, not without its difficulties, the policy for change is expected to be one of gradual migration rather than a 'big bang' replacement approach. Significant areas of functionality will be removed from existing mainframe 'hub' systems which will be re-engineered into the role of major data servers. (Major systems, whether legacy systems or new strategic components, are referred to as 'hub' systems in BT OSS terminology.)

One further constraint is that it is BT's technical policy to favour open systems solutions wherever practical.

1.2 Aims and principles

Process and documentation

CHIS grew out of the unified customer handling (UCH) project, which was the culmination of a process seeking to develop an architectural framework for the customer-facing aspects of BT's OSS. UCH's origins were in the work done with MCI and Concert to agree principles for interworking in the Global Customer Service domain. These principles were then enhanced and refined by BT in the context of its UK national operations and the overall OSS 'tile' architecture [1].

One of the fundamental issues of 'architecture-led' developments is that there are potentially as many definitions of the meaning of the term as there are architects. At least one definition of the architect's role includes the writing of C header files. The scale of the re-engineering enterprise implied by UCH and its predecessors was considered by the core team involved to preclude a centralised architectural role of this nature. UCH and CHIS have, in practice, progressed fairly effectively without ever reaching a complete consensus on a formal definition of 'architecture'.

The core thinking behind UCH and CHIS is expressed in five major documents and regularly enhanced, glossed or revised in supplementary presentational material, or in more detailed design standards. The five key documents in effect represent the perspectives of five key interests or stake holders.

- The UCH technical architecture — this defines the main technology choices and something of the principles of how the technology is to be used. In comparison with real architectural activity this is equivalent to drawing up the Building Regulations and a list of BSI or Agreement Certificates together with some structural engineering principles. This was, although controversial, in some ways the easiest statement to make and is certainly in practice the most commonly referenced.

- The UCH functional architecture — this attempts to define the business functional domains and also to define in principle how these domains map on to the ideal physical system architecture. It might be compared with a town plan. The functional architecture was expressed in object-oriented terms, but this led to some unproductive debate and some confusion arising from the more commonly adopted use of OO principles at a much finer level of granularity and during actual software development. The functional architecture does not attempt, in general, to define physical implementations or to define an evolutionary path.

- The CHIS security policy — BT now requires the production of a security policy document for all its major OSS systems. This covers all aspects of security including:

  — software provision for authentication, authorisation, integrity and privacy,

  — system administration and operational procedures,

  — development and configuration processes and environments.

- The UCH management architecture — this document outlines the approach to system operation and administration, recognising that these functions need to be represented by a meta model overlaying the whole system.

- The CHIS network architecture — this is a description of the assumed network and processor configuration and capabilities on which CHIS expects to deploy.

It is probably true that the process of production and review of these documents was of more value than the documents themselves as a subsequent reference source. The value of the process was in developing a working consensus, or at the least a pool of common ideas and vocabulary which enabled useful further progress and provided a basis for publicity, educational and marketing activities within the rest of the BT development community. This experience tends to confirm the importance of architecture as a role or project activity as much as formal blueprint.

Downstreaming the architectural vision to the level of construction commonly presents difficulties. These have been compounded in the BT OSS case by the scale of BT as an organisation, the existence of organisational boundaries at the design fault lines and the adoption of an internal market approach to the development process.

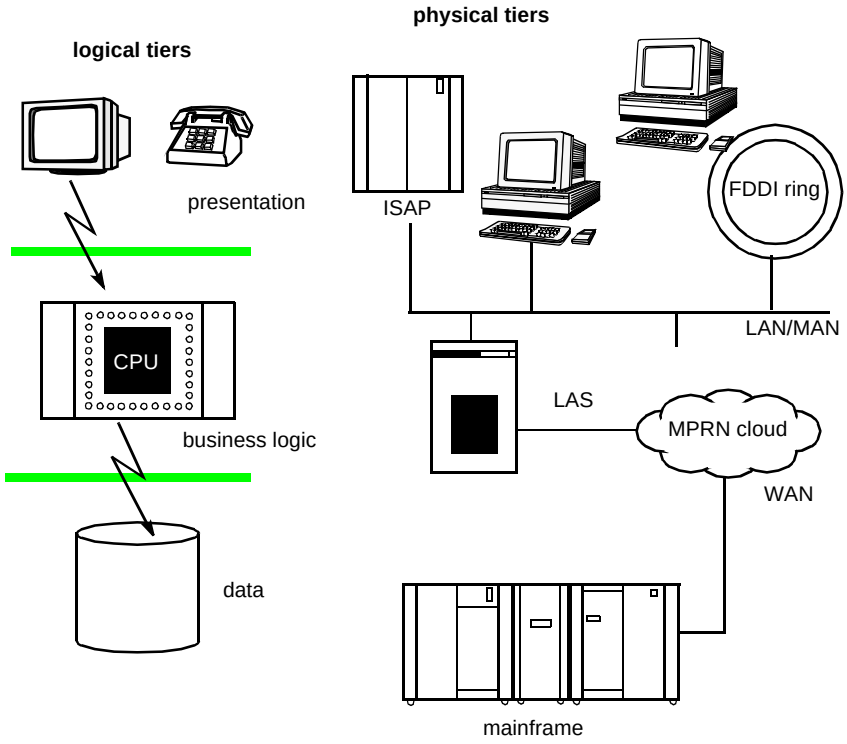

Fig 1 The 'tiers'.

Three-tier architectures (3TA)

The UCH architecture can be summarised as a 'three-tier, service-based architecture'. It was recognised quite early that it was necessary to distinguish two senses of 'tier' but some confusion continues particularly in relation to the meaning of 'mid-tier' services. Figure 1 illustrates the two senses of three-tier — the logical and the physical and how they are related.

A question which is commonly asked is how the logical tiers should map to the physical tiers. A simple answer often given in the context of client/server computing is to consider a matrix based on frequency of update and frequency of retrieval of data. The UCH perspective on business logic and data placement in the physical 3TA is illustrated in Figs 2 and 3.

It is part of the UCH/CHIS credo that the 3TA is only a stepping stone towards the real end game of open distributed computing. Even if the three-tier model is to be followed quite strictly in relation to new developments, it is clear that in evolving BT's OSS there is a need to deal with

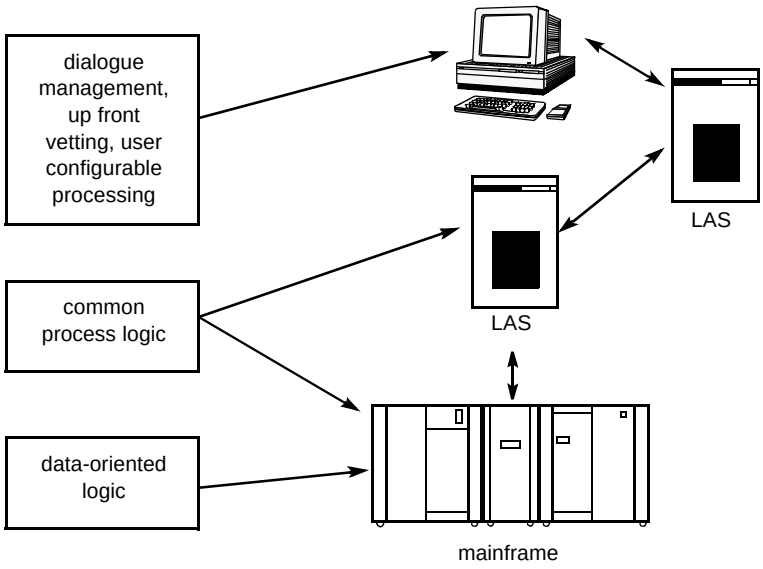

Fig 2 Business logic.

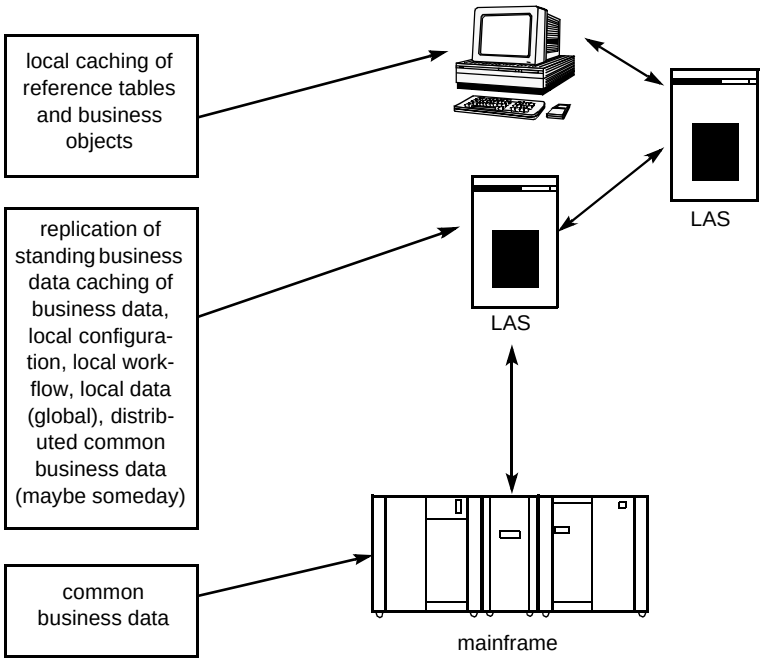

Fig 3 Data persistence in the three tiers.

a federation of systems, some monolithic, some two-tier, and some three-tier in their own right.

A more sophisticated set of concepts is required to described the various possibilities of a three-tier or an 'n-tier' architecture, together with a larger set of criteria which include a range of non-functional attributes. Rather than considering business logic and data independently, these principles need to be expressed in object-oriented terms. Factors to be considered for 'object placement' might include:

- scope of use — individual, local workgroup, regional, national, international,
- scope of policy relevance — individual, workgroup, channel-specific, enterprise-wide, external,
- security classification,
- object size,
- frequency of use,
- impact of loss,
- persistence characteristics,
- tolerance of variation (i.e. how important is it for all copies to be always in step?),
- object character (i.e. is it an interface object, control object or entity object or some hybrid?),
- end-user access requirement (timeliness and synchronicity).

All these factors must be matched against physical network, processor and device capabilities and costs and practical constraints, such as available configuration management capabilities.

The CHIS project's view of its scope is that it needs to deliver services which will typically have the following characteristics:

- appropriate for use in a national, or international, enterprise-wide context,
- with security classification of up to 'In Confidence',
- with message sizes typically of a few hundreds of bytes, exceptionally up to a few hundred kilobytes,
- with a medium-to-high frequency of access typically with a medium-to-high retrieval-to-update ratio,
- with moderate impact of failure for an individual transaction but a very high impact of failure on the service as a whole,
- will often be stateless or required to be persistent during the course of a specific client/CHIS dialogue,
- have some tolerance of variation.

Subject to some performance and other constraints, these characteristics are, in general, favourable to both logical and physical mid-tier placement of CHIS services.

2. CHIS outline description

2.1 Other BT client/server systems

BT has implemented a number of client/server systems, most commonly two-tier — either Windows PC clients screen-scraping legacy-MVS systems, or stand-alone two-tier applications based on SQL*Net, or ODBC-based interactions between a PC client and a relational database.

Five examples are worth considering to better illustrate the CHIS rationale (COSMOSS, ServiceView and SMART are described in detail elsewhere [2—4]).

ServiceView manager

This system, which is one of the planned CHIS clients, exists in a number of variants and is deployed in global and major customer-handling channels, within the major customer service system (MCSS) programme. This is based on an Oracle forms client and an SQL*Net interface from client to server. However, unlike the customer service project (CSP) (see below) it permits access to data only via stored procedures. There is no clearly identifiable logical mid tier, however, and CHIS access is handled via a gateway daemon process supplemented by screen-scraping hub systems.

SMART

SMART is being deployed in Personal Communications volume customer service channels. It is primarily a C++ application running under Windows NT which front-ends CSS using the middleware message-based interface (MMBI) . It has a physical mid-tier server which provides a replicated product database and a communications gateway. Both ServiceView Manager and SMART predate CHIS and are being adapted to migrate towards the approved form of 3TA.

Multi-service billing front office partition (BFOP)

The BFOP is a Windows PC client communicating with a local Unix server via DCE RPC and via BT and COSMOSS middleware to a DB2 database layer. It is therefore nearest in principle to the proposed CHIS architecture. It is at variance, however, in using DCE only as a transport mechanism without exposing fully defined and typed interfaces on the physical mid-tier server. Interface definitions are held in a case tool and used to generate stub code for the client and at the database layer.

COSMOSS

COSMOSS is an MVS DB2 system which supports order entry and provisioning for special (non-PSTN) services. It is the most recent of BT's live mainframe OSS systems and has a 3270 user interface. It has three characteristics which make it worth commenting on here, however. Firstly, it shares with CSS the common CICS middleware which, in a limited way, separates screen handling from application logic. Secondly, it has a data encapsulation layer based on an additional piece of BT middleware called XCALL [5]. COSMOSS may therefore be regarded as having a form of logical three-tier architecture, and was designed as a 'distributable' system, albeit in the context of a homogeneous proprietary platform.

VBS Telesales

This project supports proactive sales functions and is based on the Versatility call centre package supplied by NPRI. It consists of an SQL Windows/Visual Basic client which interacts with a local RDBMS via ODBC and in BT's CASE Oracle SQL*Net. Interaction with CSS is planned to be asynchronous via a daemon process which reads database tables and calls CHIS services. The second release of VBS will call CHIS synchronously from the desktop for functions such as appointment making which must be handled interactively, using Gradient DCE client software running under Windows NT. The Versatility client application also uses DCE to call its telephony server.

Customer service project (CSP)

This system is a customer-handling and order-entry system deployed within BT's managed network services (MNS) organisation. It is superficially a three-tier architecture with a Windows/Visual Basic PC client, a Tuxedo-based mid-tier service layer and a legacy-system interface. However, it is more accurately described as logically a two-tier client/server system with a gateway process as there is no encapsulation layer between the Tuxedo services and the local relational DBMS, and the legacy interface is handled essentially asynchronously, via database queues and a daemon process.

CSP is also of interest in presenting an event-based API to the GUI developers via some ICL-sourced middleware.

2.2 CHIS deployment scope

Figure 4 illustrates how some of the developments identified above are planned to interact via CHIS services. CHIS is seeking to consolidate and rationalise all access to legacy systems via a single remote procedure call (RPC) based API using the Open Group's DCE middleware technology.

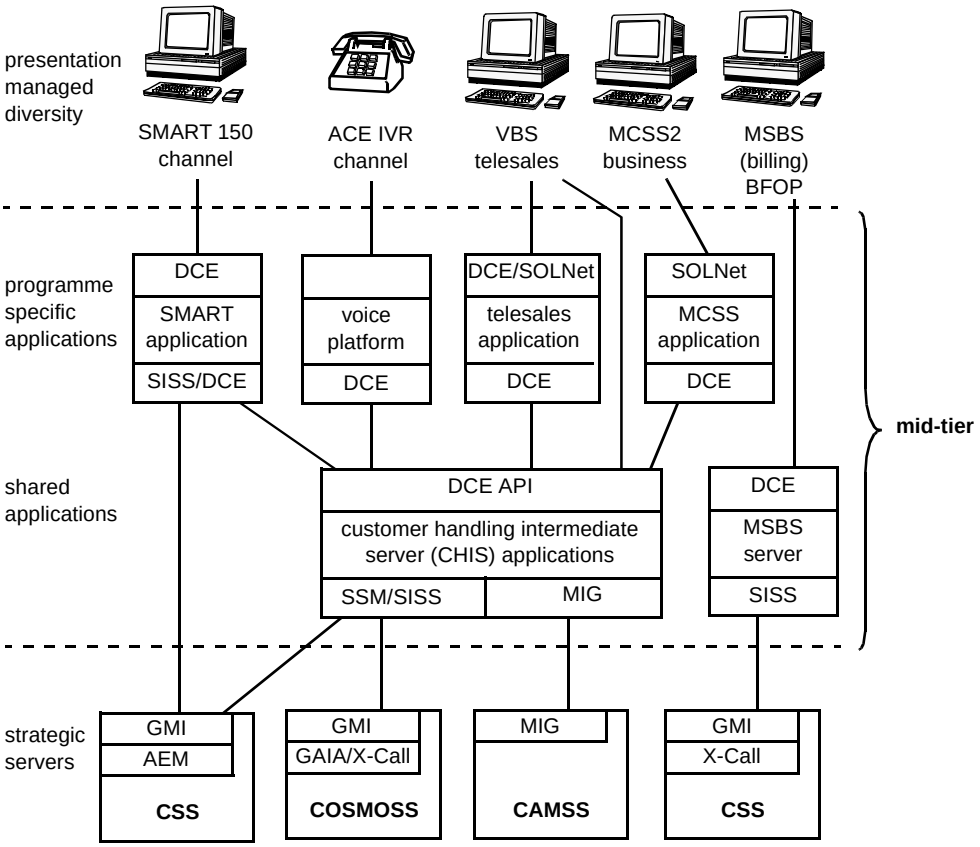

Fig 4 Overall outline of planned CHIS deployments.

2.3 Service examples

There are five CHIS RPCs currently deployed in live use by the ACE (Automated Customer Interface) project.

There are two business functions supported by the RPCs developed for ACE. One of these is concerned with automating the process of acknowledging a customer call in response to their receiving a 'red reminder' and is not discussed here. The other three support a simple order-entry service used initially by BT shops and third-party retailers to enter network service orders and billing options for existing customers. The latter service will be reused almost immediately by the VBS telesales application and is illustrated in Fig 5.

To provide the service three RPCs are available:

- check retail status,
- check product compatibility,
- enter simple order.

The first of these checks whether the customer account is in good standing and a working installation exists for the supplied telephone number. The implementation of this RPC combines data retrieval from CSS via an MMBI script driving existing CSS transaction modules, with some additional logic coded in C++ running within the DCE server.

The second RPC checks whether the requested product is technically capable of being provided on the proposed installation. This breaks down into checks on serving-exchange technology, access-line technology, compatibility with existing products and services, compatibility with products or services on order, and also whether the product is appropriate for the class of account.

Some of the underlying logical operations are likely to be useful in their own right for later applications but are not implemented as exported RPCs at present. In fact, for performance reasons, as discussed later, the logic dealing with network-service product-compatibility checks is implemented in new COBOL modules in CSS, invoked by a new middleware facility.

The order entry service simply drives existing CSS transactions (enter order, enter order line and enter order details), providing default values for many data items not relevant to the simple-order function. It also provides some overlay logic driven by local configuration files which, for example, translates certain product codes implying a free trial period into two orders with the appropriate parameters.

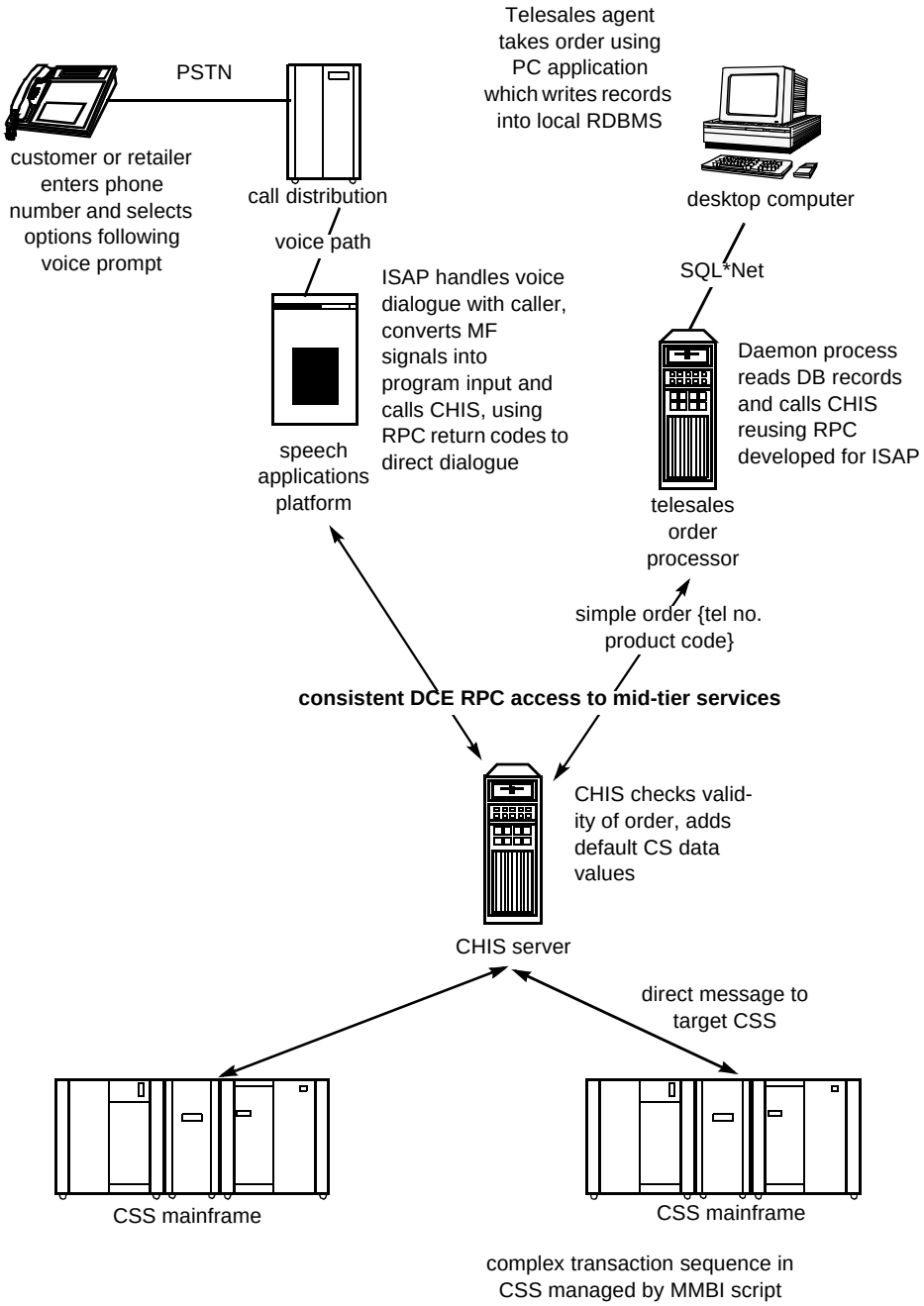

Fig 5 Example service — simple order service.

The service overall is designed so that a client may call all three RPCs in sequence. Context is maintained by passing a token within a private data structure, not normally updated by the client application programmer, indicating that this has been done. It is also possible to call only the order-entry RPC. If this is done without having previously called the first two RPCs, as indicated by the absence of a context 'cookie', they are called automatically by the order-entry RPC prior to placing the order.

The ACE client in practice calls the three RPCs individually as this is necessary to effectively support the customer dialogue on the ISAP. The telesales client which calls this RPC asynchronously does not require to do this.

2.4 Security policy

Like all major systems in BT, CHIS is required by BT corporate policy to have a security policy document and to go through a security accreditation process. The prescribed format and process were developed in the context of traditional integrated systems where it is straightforward to identify system boundaries and system ownership. This led to some technical difficulties in drafting a policy.

The CHIS security policy cannot stand in isolation but must be reviewed and correlated with security policies for those systems with which it must interconnect. It must allow

for the definition of mutual responsibilities and boundaries with several classes of system or service:

- desktop client systems under the direct control of an identifiable end user (e.g. VBS PC client),
- proxy servers providing services to end users but acting as clients of CHIS:

  — where the end user is an identifiable internal BT user (e.g. MCSS/ServiceView in CSCs),

  — where the end user is an identifiable external user (e.g. external users of MCSS),

  — where the end user cannot be fully authenticated (e.g. ACE),
- hub systems, i.e. major BT systems such as CSS, COSMOSS and CAMSS which are repositories of commercial data,
- the BT data networks which it uses,
- master DCE directory, time and security service,
- the enterprise systems management framework (ESMF).

CHIS itself will in some cases act as a client, e.g. with some forms of service involving event notification the interaction with the client system will be initiated from CHIS.

At a general level the policy states the following.

- **Identification and authentication** — any person or user accessing CHIS from a desktop client system shall prove that their claimed identity is legitimate, such that the system may place confidence in that claimed identity.

  Where CHIS is connected to external systems, they shall prove that their claimed identity is legitimate, so that the system may place confidence in the claimed identity. All users and entities that access the system shall be uniquely identified.
- **Access control** — access to CHIS resources shall be restricted to only those users and connected systems that have been authorised and authenticated for usage of the resources by the approved communications paths and access methods used.
- **Accountability and audit** — CHIS users and CSS, CAMSS, COSMOSS, MSBS, ACE ISAP, MCSS server, VBS server shall be made accountable for their actions. Audit trails of user and interconnected system actions shall be maintained such that unauthorised CHIS activity will be detected.

Because of the special nature of CHIS within the new forms of client/server architecture, the above principles were further elaborated as follows.

- For service requests originating from proxy servers CHIS will require that, where possible and meaningful, the proxy server passes the identity of the individual end user as part of the service request. The mechanism for performing this function must be secured against the relevant threats.
- CHIS must agree with hub systems a mechanism for maintaining an audit trail for the end-user identity through to hub systems where this is necessary to maintain audibility of, and accountability for, system use.
- The interconnection agreements between CHIS and its clients will specify whether the client system can be accepted as providing a valid and trusted authentication of its end user either through its own mechanism or through a common DCE security regime. All agreements will depend on the existence of an appropriate and accredited security policy and implementation for the client. If these are in place and the client system is under BT control, CHIS will accept the client's authority for the authentication of the end-user id passed by the client. It will not be required that CHIS and/or the hub systems reauthenticate or independently authenticate the end user. A similar interconnection agreement will be agreed between CHIS and the connected hub systems.
- In the case where authentication can only be relied upon as far as a third-party retailer agent or other licensed operator (OLO), further counter-measures may be required to ensure that misuse of services by the external agent can be detected.
- Where, because of the nature of the business process involved, it is not feasible for the CHIS client or the common security regime to provide a strong or effective authentication of the end user, the client's access to CHIS services will be limited to services where the impact of a loss of accountability for the use of the service is not significant. This will be subject to review on a client-by- client and service-by-service basis. Procedures will be required to ensure that enhancement of service implementations does not invalidate previous connection agreements.

CHIS is atypical in stressing to this degree the formality of interconnection agreements with its clients and supporting servers. The interconnect agreements also serve as Service Level Agreements for assessment of operational performance and are based on service definitions for each CHIS service which equates generally with a specific DCE RPC.

2.5 Service Definitions

CHIS intends to make a major feature of providing consistent, useful and formal definitions of its services. In contrast with most in-house developments where documentation, if produced, tends to define how code has been implemented and is poorly maintained, CHIS service definitions are proposed to be among its key deliverables.

Service definitions will be used to:

- advertise details about the CHIS service to actual or potential client projects,
- provide reference in interconnection agreements with client projects,
- support identification of opportunities for reuse,
- provide a base input to CHIS VV&T,
- provide reference for CHIS support functions in assessing problem reports and monitoring CHIS service levels.

3. CHIS design principles

3.1 Software engineering principles

It has been proposed above that the solution to the trade-off between speed of response to business changes and the costs of system proliferation is through the large-scale application of software engineering principles. Booch [6] identifies seven elements of object-oriented design, four of which he categorises as major and three as minor:

- major:
  - — abstraction,
  - — encapsulation,
  - — modularity,
  - — hierarchy,
- minor:
  - — typing,
  - — concurrency,
  - — persistence.

Several of these have been identified by other authors either as basic principles of software engineering or in slightly different forms.

Abstraction and encapsulation

CHIS began with the intention of emphasising a high degree of abstraction and encapsulation in its service interface definitions. In practice, this has not been easy to achieve in all cases.

CHIS has not had the luxury of being able to develop a comprehensive class model which is suitable for direct implementation in software. The UCH functional architecture was intended to be the starting point for this and was to be refined through definition of detailed domain specifications. However, this has only been done for a very limited number of domains and only in one case with any high degree of credibility (the call-reception domain which is largely concerned with computer/telephony integration).

There appears to be a very substantial semantic gap between the concepts appropriate to modelling business processes (at least as implemented in BT's legacy OSS and associated procedures) and a class model which accurately reflects code constructs used at an implementation level. This is very evident from the complexity and obscurity of the class model reverse-engineered from the SMART Visual C++ application. In practice, however, many of the DCE RPCs invented in CHIS could be mapped to a logical operation of an identifiable business entity.

The difficulties in constructing and maintaining a logical model seem to arise from the qualitative difference between the overall business analysis and the software construction perspectives. Programming staff have favoured an iterative approach to reuse. That is to say they have lacked any credence in any overall analysis of the customer-handling problem space and preferred to focus on specific use cases, proposing to expand analysis over time. Extension of an interface definition beyond the specific first-stage requirements with a view to making it more generic, and hence more reusable, has also been resisted.

The danger of this approach is that CHIS will become a collection of obscure individual services with much duplication and a failure to protect its clients from underlying changes. Once services have been deployed it becomes difficult to persuade clients to replace them with more sophisticated successors and CHIS might build up an impossible maintenance and support overhead.

Even when formal 'use case' techniques have been adopted and a wider scope of requirements has become available in the second stage of the project, further difficulties in following an architecture-led approach have been encountered — see the discussion below in section 4.1.

Modularity

The modularity of CHIS at this stage looks to be potentially sound. Physically at run time it takes the form of DCE server builds, but at a base-configuration management level each server is composed of multiple components. All servers contain one set of standard infrastructure code and its associated management interface, regarded as a single component for configuration management purposes and one or more application components, each with a single DCE interface. At this stage of the project the components must be statically compiled and linked into the servers.

Hierarchy

CHIS has not defined a strategy for programmatic implementation of inheritance in business application code at this time, or even addressed in a co-ordinated way the standards for implementation of server application code internally. It has aimed at achieving reuse at a higher level of granularity (i.e. at the component level, as described in the CHIS software model in section 3.3) on the assumption that if this much reuse of code could be achieved, very satisfactory gains could be made anyway. Reuse at a lower level (with the exception of common commercial C++ class libraries and infrastructure code) was expected to be difficult to achieve given the organisational problems identified below.

This approach is possibly not sustainable in the medium term if the maximum possible productivity and maintainability is to be delivered.

Already there are indications that independent developments of variants of the same logical functions are being embarked on to meet specific client needs in varying time frames. This would be most sensibly addressed by rolling forward the previous version, of code into succeeding versions, but to do this effectively across a diverse and geographically diverse set of developers in the absence of a comprehensive and detailed physical software architecture is challenging.

Arguably, in other ways CHIS has correctly adopted the underlying principle in its vision of the logical 3TA and its internal layering of application code on top of common infrastructure.

Typing

CHIS has adopted a policy of favouring precise, statically typed interfaces defined in DCE IDL, holding out against requests for loosely defined or totally generic interfaces which are dynamically interpreted by the client.

It is considered that this policy is well calculated to sustain operability, resilience and consistency within the 'hostile' distributed environment in which deployment occurs.

Such a policy creates tensions as there is a general desire to make progress in development without having sufficient certainty to make firm decisions about interface content.

The CHIS view also sometimes appears in conflict with a vision of resolving the time-to-market issues identified in the opening sections of this paper by building systems which are highly configurable or 'data driven', in the sense that system operations are highly abstracted and generic (and hence stable) but adaptable by run-time configuration parameters.

This is actually somewhat orthogonal to the design themes expressed in CHIS, and is an issue of dynamic definition of process more than dynamic interpretation of data. While it is clearly a sensible approach for some purposes and at a certain level of granularity, taken to its limits it can produce systems which are very complex to operate, still require developer support to amend configuration data, and need as much regression testing following a change of configuration as more traditional applications following a code amendment. It is anticipated that once sufficient CHIS services have been developed they may at some stage be linked into larger automated processes through deployment of workflow/process management technologies, when these are mature.

Concurrency

The basic mechanism employed to express a concurrent process in the CHIS mid tier is a DCE process thread. At current levels of usage it has not been necessary, for performance reasons, to create multiple process instances. Thread creation is transparent to the DCE RPC client. Multiple-server processes are created for resilience and in the case of the ACE client a simple round-robin load balancing strategy is implemented in the client.

The effectively stateless nature of the current services makes management of concurrency in CHIS straightforward at present.

Persistence

CHIS relies on its underlying hub systems for persistent storage of business data. CHIS server configuration data is held in local Unix filestore and is read into memory at server start-up. Configuration data may be reloaded in flight through a call to the appropriate method in the management interface. The current services do not require to update data in more than one hub system and the processing model is synchronous. As there is no end-to-end transactional integrity, it is possible (but unlikely) that a CHIS service

may successfully update a hub system without the client application receiving a response to the RPC. At this stage CHIS clients that call CHIS asynchronously to the actual end-user transaction are expected to provide their own exception logs and recovery procedures to cover this possibility.

Asynchronous operation between end users and the CHIS service creates another potential problem. Database locking is handled by the hub-system modules and is 'optimistic'. Characteristically, in a multi-phase transaction no locks are applied via the DBMS in the read stages of a sequence, but each record in an IDMS database or each row in an RDBMS has a record update number embedded in it. Transactions retrieving data for potential update maintain all such values in store, though they are not displayed on a screen. When an update phase is reached the relevant records or rows are locked, and a standard piece of code is called to check the stored record update number against the current value on the database. If the values of any of the items checked differ, the update is rejected. In a legacy system the application overall will have been designed to ensure that the end results are consistent with terminal user expectations.

In the case of the MCSS and telesales client CHIS applications, updating transactions (typically amending customer records, entering orders or fault reports) are prepared against locally held copies of the data in the hub system to be updated (CSS, COSMOSS or CAMSS).

These copies may be obtained via some intermediate process such as a marketing analysis system and may be some hours or days adrift in time.

It is undesirable to require such clients to attempt to maintain any notion of record update numbers since this would, as it is implemented, require a more-or-less precise knowledge of the underlying hub-system physical-database schema. To achieve comparable results to the legacy application interface it would be necessary to compare, item by item, data values stored in the client, and, potentially significant to the validity of the transaction, against their equivalents held in the hub system before permitting the update. This implies re-engineering of the legacy applications which may not be practical.

This does not mean that the database can be corrupted, merely that the business effect may, although valid in system terms, be at variance with that intended. The problem may be more theoretical than practical and would largely disappear when all updates for a particular set of data originate from the same client. The first two releases of CHIS have not encountered this problem due to their limited scope, but the next major release is likely to have to address it. Counter measures are likely to be specific rather than general.

3.2 DCE

DCE was originally selected as the technology for synchronous client-to-server and server-to-server communications for the following reasons:

- it was non-proprietary and therefore acceptable to partners and implied no lock-in to a specific vendor,
- it provided comprehensive security features,
- it was very scalable through threading and server replication,
- it supports a service-based architecture with a possible migration path to object-oriented technologies,
- there was potential for integration with BT's existing use of IBM MVS systems and the RACF security product,
- it supports loosely coupled connections between co-operating organisations with minimum need for common administration,
- it was readily extensible through addition of new interfaces and server modules.

At the time the choice was made the alternatives considered alongside DCE were Oracle SQL*Net and associated Oracle products, and Tuxedo.

Support for transactional semantics and two-phase commit protocol was not considered sufficiently useful to be a qualifying attribute, since the lack of support for this in almost all BT's legacy applications meant that it could not generally be made use of, and in practice it is rarely essential, or causes more practical problems than it resolves. Multi-threading and/or server replication in DCE were considered to provide adequate support for concurrency without requiring traditional TP monitor capabilities. It was considered that a TP monitor such as Encina could be added to the architecture at a later stage if this seemed appropriate.

These considerations appear so far to have been well judged. Although the project has suffered some costs in terms of the incompleteness of DCE in regard to usability, programming support and operability, these obstacles have now been largely overcome by internal infrastructure development or are being addressed either by the Open Group or independent software vendors.

3.3 CHIS software model

Figures 6—8, in Booch notation, represent an abstract software model of CHIS.

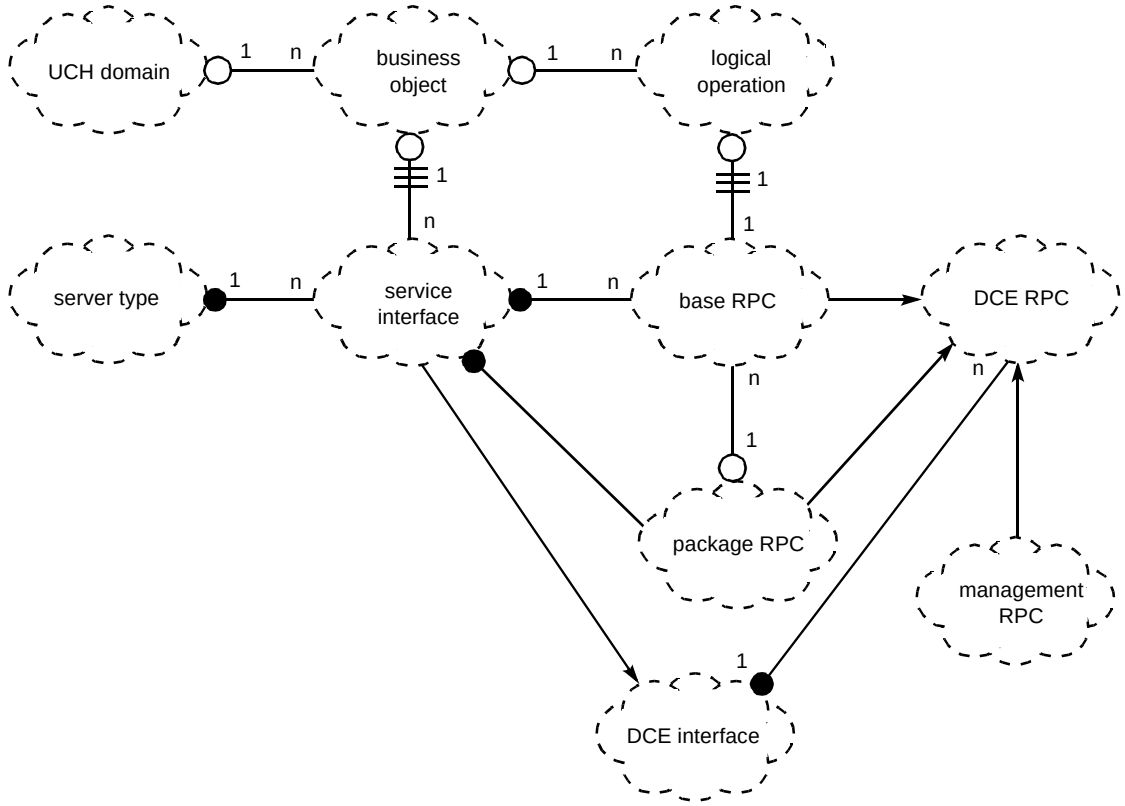

Fig 6 Logical to physical mapping.

Figure 6 shows how the logical model derived from the UCH functional architecture is mapped to the logical elements of DCE.

Figure 7 shows how a CHIS DCE server is physically constructed from base software components and Fig 8 shows the associations between CHIS software components

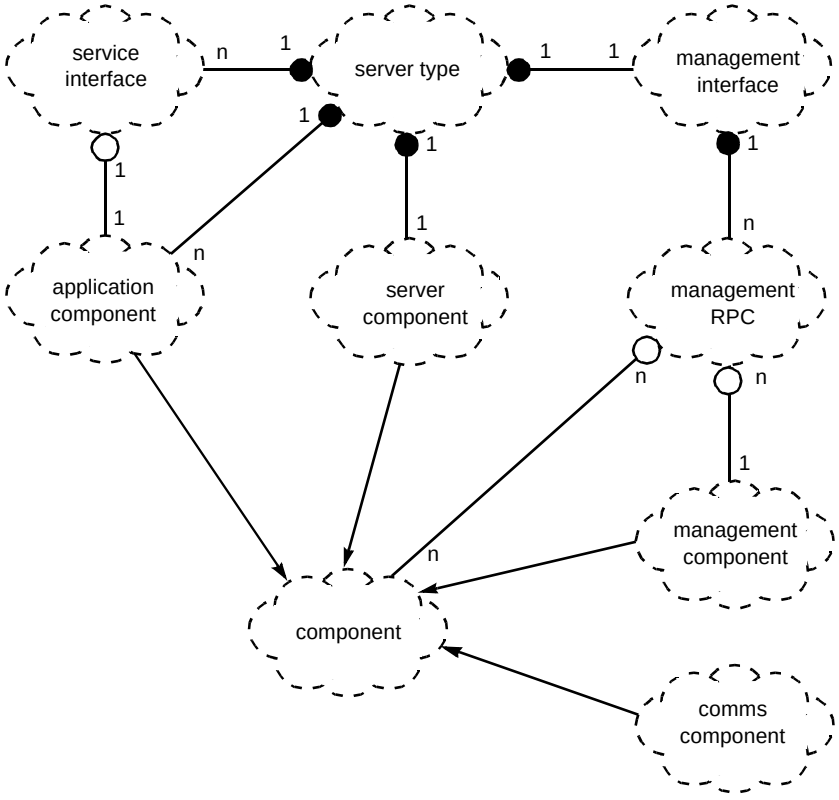

Fig 7 Server construction.

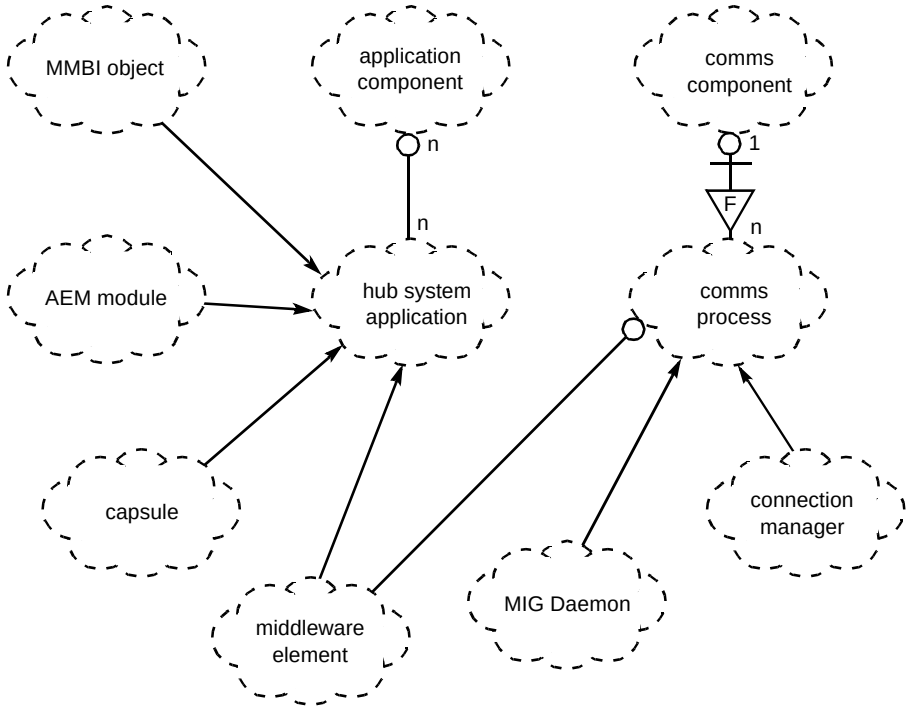

Fig 8 Legacy system links.

and legacy system modules in the three primary BT legacy systems.

Logical modelling concepts

UCH domains, business objects and logical operations are all logical modelling concepts which are documented in the CHIS service object model. Other elements in Fig 6 are implemented as software.

Software elements

There are six primary software elements and these are listed below.

- Server type

  A managed and version-controlled set of code components which when compiled and linked will run as an independent process (potentially with multiple interfaces) on a CHIS platform and support one or more service interfaces — there may be many instances of a server on single or multiple CHIS machines.

- Interface

  A DCE IDL interface.

- Component

  A component is an element manageable through a server's management interface. A component may have one or no service interfaces. A minimal CHIS server would consist of one service interface with an underlying component plus infrastructure and management interface.

- Communications (comms) process

  A separate process running on the CHIS platform which is dedicated to providing communications services to other systems, but does not form part of the basic operating platform — some such processes may be DCE servers in their own right. MIG Daemon and Connection Manager are two examples of such processes. The former communicates over a TCP/IP protocol link to a legacy IDMS/DC system and the latter via an LU6.2 link to MVS/CICS systems.

- Hub system application

  A version-controlled software element within systems such as CSS, CAMSS and COSMOSS with which elements of CHIS must interact to provide CHIS RPC services — such elements may be composed in their turn of further elements which need to be separately version controlled within the hub-system configuration management system. For example, an MMBI object will use one or more CICS transaction modules which are independently controlled. However, so far as CHIS is concerned it seeks to manage compatibility against the controlling object in the hub system.

- Management interface

  A DCE interface dedicated to the provision of standard management functions — all CHIS servers will support one management interface which will normally be the same in each case.

3.4 Naming and associated standards

Although potentially viewed as a relatively trivial matter, appropriate and consistent naming standards are probably vital to the long-term success of CHIS. Their application is the most general and effective way of ensuring that significant elements of the architecture are actually implemented by the developers.

CHIS has promulgated standards covering the following elements of the system:

- DCE interface names,
- DCE RPC names,
- data types in IDL definitions,
- exception code numbering,
- server identification.

All standards are interrelated and for the most part relate to the basic concepts of application domain, as defined in the UCH functional architecture, and of component, as described in section 3.3 above.

3.5 Stateless servers

CHIS has adopted a policy of making its servers stateless in order to simplify development in its early stages and improve resilience characteristics at the mid-tier server level. This is sustainable so long as CHIS services are concerned with wrapping hub systems. Some maintenance of context is supported across service calls for performance optimisation purposes. This is done by passing context data (sometimes referred to as a ‘cookie’) in a standard data structure within each RPC and not by DCE context handling which would have more limited scope. The descriptions of the ACE order entry service gives an example of how this is used. Another example is the optimisation of transaction routeing by passing the target CSS system identifier once this has been established in the initial RPC.

3.6 CHIS advantages

Operability

CHIS sets out to support comprehensive remote operations at a component level. This topic is discussed further by Winton [7].

Mid-tier routeing

CSS was originally designed as a district-based system. BT’s customer-handling channels are now organised nationally by market sector and function so that users require access to all 29 CSS system instances. This is achieved by ‘transaction switching’ — a facility whereby transactions are inspected in CSS CICS-based middleware and can be onward routed to a remote CSS system if the key value is determined to belong to another system.

In CSS transaction switching, resources are held in both CSS systems for the duration of the transaction. CHIS allows a single-step routeing to the correct CSS making a small contribution to easing the performance issues on the larger CSS systems.

CHIS also allows for pooling of SNA sessions into each CSS system obviating the need for each client system instance to have its own CSS session. CHIS also extends the routeing capability to systems other than CSS, or even permits transactions on more than one legacy system instance to be combined into a single service, though this has not in practice yet been required.

Consistency

CHIS provides a single protocol, and interface style to its clients irrespective of the nature of the underlying hub system. This will be of benefit where a client needs to access more than one hub, which will become increasingly likely with successive re-organisations and hub system changes. It is also a benefit to client programming productivity since developers do not have to learn a different style when they move on to new projects. A single security model is also implemented.

Maintainability

CHIS aims to provide its clients with a properly defined and fully supported service. Although robotic applications have used CSS in particular through HLAPI access to native 3270 screens (‘screen scraping’), or through the MMBI facility, which at least protects client applications from some CICS screen changes and can improve performance characteristics, such use has not been supported by a proper service definition. CHIS will seek to present a simplified

and abstracted interface, and define and maintain performance characteristics.

Reuse

The strongest justification for CHIS is that services developed for one client can be reused by others either unchanged or with minimal change. Beyond this, once a catalogue of services has been developed, new clients can be developed to meet the needs of specific user roles or processes with minimum delay and at minimum cost. CHIS terms this form of development ‘plug and play’. The value of CHIS in this respect goes beyond simple software reuse, since it comes also through its creation of a common services support environment which provides testing, integration and migration facilities, configuration management and deployment support in a co-ordinated way, with considerable economies of resource and enhanced quality.

4. Practical problems

There are a number of problems to be faced and these are divided into two classifications — technical and organisational.

4.1 Technical

Network bandwidth and latency

The physical constraints of the current network infrastructure must be taken into account in CHIS service design and tend to work against an ideal solution from the perspective of delivering fine atomic reusable services.

Clients are typically connected to CHIS servers via an Ethernet LAN or a LAN plus an FDDI MAN. In the worst case, particularly in a fall-back situation, the connection will be via BT’s multi-protocol router network (MPRN). This network is continually being upgraded, but service designers are currently constrained to assume a mean network latency of 0.1 sec between client and CHIS.

Connection between CHIS and MVS hosts is via several layers of software and a minimum of two gateways and communications processors. Internal host response time is typically of the order of 1 sec so that the end-to-end response time for the originating client cannot be guaranteed to be much less than 2 sec.

Multi-threaded/asynchronous versus packaged interactions

An end-to-end service response of 1—2 sec, which is typical of CHIS services to date, is acceptable for many purposes but becomes unacceptable when the end user dialogue is held up pending a sequence of such interactions.

This would frequently tend to result from the development of atomic services at the CHIS level.

One potential solution to this issue is to spawn multiple threads in the client and issue simultaneous calls to CHIS, and/or handle responses asynchronously. In practice, this is often unacceptable to the client developers due to the additional complexity, and hence development cost, imposed, or it may even be technically impracticable due to the limitations of the client platform (Windows 3).

CHIS has attempted a partial resolution of this issue by distinguishing its services as ‘basic’ and ‘package’ services. The basic services are those services which are identified as sufficiently atomic and generic to have major reuse potential. Package services are services assembled from basic services to support a specific client requirement. The interaction between client and CHIS remains synchronous, but the overall response time is reduced as only one call needs to be made over the network.

Sims [8] recommends an architectural approach based on an event-driven API model between clients handling user presentation and the business services layer, rather than the synchronous RPC currently offered by CHIS. This is valid in the context of a GUI which requires to interact rapidly and flexibly with mid-tier services.

The CSP project (see section 2.1 above) used an ICL product, then marketed as dialogue management services (DMS), to provide the event-driven interface between its Visual Basic presentation application and the underlying Tuxedo calls. This was technically quite sound but proved difficult to fully exploit in practice. When it was desired to maintain transactional integrity through Tuxedo calls, the norm for updating services, the client application had often to be blocked in any case, as the Open TP standard effectively prohibited multiple simultaneous transactions being launched from a single client. As CSP derived much of its data from a local Oracle DBMS and had only a single legacy application with which to interact, mainly in update mode, the potential advantages of the DMS event-driven approach were therefore minimised.

CHIS has a distinct development strand concerned with event processing (not otherwise discussed in this paper), but this is concerned with events at a much larger level, that is to say business events concerned with monitoring service provision processing and customer service processes over periods of hours or days. CHIS has, therefore, yet to properly address the issues on which Sims focuses [8].

CHIS faces comparable issues in its transactions with the legacy hosts. Although in the CHIS environment multi-threading is practicable, the adoption of this approach is likely to result in an additional and unacceptable-performance demand on the host system, due to multiple

transaction overheads and unnecessary data access where there are multiple logical paths which depend on intermediate results.

The consequence is that in many specific cases it is optimal to further package the interaction with the legacy host and to develop the service logic in a host module. When this is done through re-engineering activity (e.g in the new modules supporting the product compatibility check RPC outlined above in section 2.3) the separation of business logic and data access capsules is observed, thus preserving the logical three-tier architecture.

Security

Existing BT systems have a variety of authentication and authorisation mechanisms. A form of single sign-on is available on many MVS systems, but, as RACF is not a consistent feature, authorisation and audit mechanisms are typically embedded in the host platform. Some new open platform client/server systems also have independent security mechanisms.

It is undesirable to require end users relying on CHIS to log on independently to all the host systems which underlie CHIS services. There are also system resource economies to be achieved by sharing a common pool of LU6.2 sessions into MVS hosts of which there are currently around 32 addressed by CHIS.

CHIS has adopted a policy of accepting authentication of end users by trusted proxy servers as valid even where they have not implemented a DCE authentication service. In such cases a secondary user id must be passed by the trusted proxy identifying the source of the original transaction request. CHIS in its turn passes the secondary user id to the legacy host system so as to preserve end-to-end auditability. The precise implementation varies from host to host, depending on whether the CHIS-to-host protocol is SNA or TCP/IP and on the host's approach to authorisation. This approach could ultimately be rendered obsolete if an end-to-end DCE security regime with delegation is adopted, with consequent simplification of user-profile administration.

Client preference for closely coupled systems

At least initially, client requirements are typically expressed in the form of very specific and detailed data elements derived from existing 3270 or equivalent screens. Existing host systems have over time come to dominate business process thinking and the tendency has been, deliberately or otherwise, to seek simply to automate and simplify the existing user interface. This tends to result in two separate but very closely coupled implementations with consequent problems of data build and maintaining consistency and a combined system which is potentially significantly less flexible than the original host.

Even when a 'use case' approach to requirements capture is adopted, problems arise because the 'use case' perspective tends to be that of the prospective end user of the client system and tends to be expressed in terms of existing legacy system transactions and processes. Time-scale pressures on development militate against the necessary second-level analysis and scenario development to produce a rationalised abstract view of good mid-tier services.

Although this problem can in part be resolved by education and persuasion of the client system designers, in some cases it is more intractable as the legacy application requires an intimate knowledge of its internal data structures for its operation, to the extent that an elegant solution may require a business process change and legacy system re-engineering as well as some adaptive logic in the CHIS services.

Client preference for package services

In general, CHIS is not in a position to impose a design on its client projects which are directly customer facing and separately funded, in some cases predate CHIS with their own architecture and design philosophy, or may even be packages with limited scope for customisation. There are often difficulties to be faced in agreeing an interface definition in the preferred CHIS style, which is to say granular, abstracted and statically typed.

In the context of 'bespoke' service development, client developers typically come to CHIS with a detailed requirement based on analysis of the existing 3270 screen-based interface. They also tend to prefer to design for the minimum number of physical message types and interactions, at the cost of designing interfaces with larger numbers of parameters supporting a mixture of functionality, which do not map neatly on to logical business objects.

An alternative approach is to seek a pipe facility which returns a data stream in type, length, value (TLV) format leaving the interpretation of the data up to the client. CHIS seriously considered the provision of an MMBI pipe facility into CSS with a view to providing an incentive to client developers to take up the DCE approach and gain at least some of the benefits in terms of the common protocol and security regime. This was ultimately rejected in favour of putting resources into the development of a service generation facility which partly automates the translation of an MMBI interface into a DCE server component with a fully typed interface.

Performance impact on legacy hosts

Certain of BT's CSS systems operate near the limits of the capability of the most powerful MVS systems, due in

part to the limited support in the underlying IDMS database software for multi-processor systems — in effect all DBMS updates have to be fed through a single IDMS central version which can only use a single processor. The CAMSS system also runs near the performance limits of its platform. CHIS must therefore be careful to do nothing to increase hub system resource usage and if possible reduce it.

4.2 Organisational

Political and commercial problems of reuse

The model upon which BT OSS development is expected to run is that of an internal market. In BT the main elements of this policy are:

- development and operational budgets are, notionally at least, held by customer-facing units or by product development,
- BT internal development resource is organisationally distributed and structured into distinct cost centres which are assessed against performance targets for cost recovery and utilisation,
- developers are in a separate organisation at director level from computer operations and from business users,
- development budgets and operational budgets are rigidly separated,
- within the development and operational split, budgets for internal and external manpower are distinct,
- there are typically at least two levels of agent between the real end users or the potential business beneficiary of any software deployment and its developers,
- all development programmes must be justified with a business case review by technical and financial committees — in practice, a strong senior business champion will usually be able to drive through a proposal against the grain of current technical policy if they can claim significant business benefit, expressed in terms of manpower reductions within the units under their control, or revenue generation or protection.

There are sound reasons behind this approach based on a history of systems developments which were sometimes perceived as owing more to developers' technical interests and prejudices than any demonstrable business benefit.

Finance systems associated with tracking the costs of systems development and operation have historically lacked precision and real credibility. It is questionable whether any BT OSS system has been able to be assessed quantitatively in terms of business benefit post implementation.

Geographically distributed resource

CHIS is constrained to find internal development resources from BT's System Engineering Centres (SECs). It is to some degree in competition with other projects in obtaining this resource.

Given their management objectives, SEC resource managers will tend to seek the kind of continuity and scale of work associated with large traditional projects and the confidence of commitment from a powerful and well-funded business champion in BT's operational divisions. They will also prefer to work in an area where they have a good pool of experience and skills.

In its initial stages in particular CHIS requires a wide range of technical skills and domain expertise, much of which is at a premium within the SECs. There are, therefore, constraints to distribute development work around several SEC units sited in London, Glasgow, Cardiff and Martlesham to obtain the necessary skills. This creates further communication problems and difficulties in reaching consensus, even within the CHIS developers, on a common approach, since to some degree developers' loyalties are to their local unit and its standards and objectives.

Alternative solutions

A proverb sometimes quoted and possibly invented in CHIS is: 'Problems which have many solutions are not easy to solve.'

Gaining theoretical acceptance of the UCH and CHIS approach was relatively easy. This was achieved through presentations to and review by the appropriate BT Design Forums. To realise the benefits, however, it was necessary also to gain practical understanding and support for its application from many players in the BT OSS market-place. Many of these had a predilection for alternative technical approaches, for example focusing on a distributed data paradigm with which many experienced DP professionals are familiar and comfortable, and distributed RDBMS and gateway technology, or favoured a more pragmatic approach generally.

In the past it has been generally feasible for individual projects to resist adoption of any corporately mandated architecture by pleading that they are merely meeting customer demand with a 'tactical' solution and will adopt the mandated technology 'later'.

Even if an approach was accepted in principle, once concurrence had been granted it was easy to subvert or ignore inconvenient constraints during development.

Of course, in many ways it is perfectly valid for projects to challenge and explore the limits of any prescribed

architecture but the challenge is to ensure that this is through a process of constructive feedback not subversion.

It is of course quite true that alternative technologies could be made to support the CHIS vision and that a specific technology choice within reasonable parameters may not be a key success factor. However, it is definitely adverse to real progress to permit a number of variant solutions without absolute clarity of scope.

Infrastructure or application?

Issues were encountered when dealing with CHIS deployment due to the sharp distinction by BT's computer operations organisation (CSO) between elements classed as infrastructure and those classed as applications.

Infrastructure includes network and computing platform resources which are clearly shared on a company-wide basis or at least in a significant way across programmes.

Anything which is not classed as infrastructure, and the definition is quite narrow, is expected to fit into a fairly traditional model of a discrete system so far as set-up costs, running costs, security and support arrangements are concerned. This means that it will have a clear business owner within a specific programme and is clearly identified as being a physical system funded by a specific programme. (CSS is, as always, something of an exception.)

CHIS fits uncomfortably within this framework as being certainly not a traditional discrete application but not falling under the traditional definition of infrastructure either. CSO's distributed processing operations programme is addressing this as well as other challenges in operating distributed systems.

5. Resolutions

5.1 Object-based architecture

The members of the core UCH team who for the most part continued to be involved in the early stages of CHIS were not experienced in the application of object-oriented design (OOD) techniques, although some had a basic theoretical knowledge of them.

It had been accepted that it would be useful to adopt a recognised methodology for the analysis and design phases. A number of options were considered, including the BT variant of SSADM and Yourdon [9] but Rumbaugh's OMT was selected as the most appropriate approach for the architectural and design stages and team members were given training in this method.

When programming teams started to become involved it was discovered that the Booch [6] method and notation, supported by Rational Rose software, was already in use in the SECs; it was therefore decided that the Booch notation be adopted as the common style. The Jakobson 'use case' approach to requirements capture was also mandated so far as practical.

The subsequent consolidation of all three gurus into the Rational Corporation and the current form of the evolving 'Unified Notation' suggest that the latter decision, if not the former, was well founded.

In the author' s view neither Rumbaugh nor Booch fully address the scope of UCH and CHIS in the form in which they were available when the project started.

Communications between the requirements analysts, the architects and the development programmers have been problematic due to the different perspectives and lack of general experience in a range of OOA, OOD and OOP techniques. The developers' perspective tends to come from training in and use of the C++ language. The UCH/CHIS core team generally come to OOA/OOD from a background in data analysis and structured methods and are more comfortable with a duality of logical and physical models. In practice, these difficulties were exacerbated by the lack of either OO experience or domain expertise in the team employed to focus on requirements.

The expression of the high-level architectural and design concepts in OO format has not always been intelligible to programming staff who expect a more precise and granular class model which is capable of more or less direct implementation as code. Within the UCH scope, a logical business object such as 'account' can never be realised as a single piece of code.

The net result is that as the project has progressed the significance of the service object model envisaged by UCH has been downplayed. The logical model should emerge as services are produced and be implicit in the naming standards which are based on the very high level UCH category/domain model. The lesson taken from this is that, while a degree of preliminary analysis and modelling is wise, development of the logical model needs to be iterative and arise from a constructive interplay between the architecture role and designers and developers. A substantial degree of patience and persistence over an extended period is required to make this work. A comprehensive and detailed logical service or business

object model cannot easily be created up front in the UCH/CHIS environment nor effectively be retro-engineered from code after a system has been developed.

5.2 Service reuse, not code reuse

Reuse can be viewed as almost a by-product of good software engineering. CHIS focuses on reuse in two major ways:

- firstly, it mandates the use of common C++ class libraries for mid-tier development (Rogue Wave),

- secondly, it seeks to provide common communications infrastructure code for each specific hub system and for management of mid-tier processes,

- thirdly, it intends to provide not just software which can be copied or linked into a variety of application implementations, but actual deployed services on the BT network.

Application reuse is intended to be managed at the level of the DCE RPC. It is not envisaged that a C++ class library for business objects will be created across software engineering centres at least in the current phase. Even at the RPC and DCE interface level some duplication is seen as acceptable provided it can be recognised and managed.

5.3 Project model

CHIS has developed the following project organisation in attempting to adapt to the demands of the internal market and in response to departmental re-organisations and loss of key personnel. From a model which assumed that the CHIS project would be directly responsible for all mid-tier service development with strong central requirements management and design functions, a more devolved approach has been adopted, to try to avoid the issue of CHIS development being cited as a bottleneck.

CHIS supports a number of different approaches to interworking with other projects.

- New clients — bespoke service development

  CHIS will develop a package of new RPCs, or a mixture of new and existing RPCs in response to the requirements of individual client programmes or projects, and arrange for the physical deployment and support of those services.

- ‘Plug and play’

  CHIS intends to provide a facility for ‘fast-track’-style developers to use existing CHIS services in prototyping, RAD and tactical or short-term developments.

- Distributed Middleware services

  CHIS provides a ‘productised’ version of its infrastructure code (for DCE server start-up, ESMF integration and/or legacy system interconnection) to any OSS project which wishes to develop services in the CHIS mould. This will normally be accompanied by consultancy in its use.

- Common services support environment

  The CHIS facilities for system integration, testing and deployment may be made available to other projects who are developing services for reuse to the CHIS standards.

  For clients and services that remain within the CHIS development project the design responsibilities have been devolved to staff in SECs. In theory there are two roles, one a client project-facing role which is responsible for the high-level design covering all interactions with CHIS of that client, and the other an application-domain design role. The intention is that specific SECs will specialise in certain application domains and in facing specific client projects. Inter-SEC co-operation is required when a particular client’s requirements include functional areas supported by another centre.

  The central design role is merely concerned with setting and monitoring standards and resolving issues of principle or major exceptions.

  With established client projects the role of requirements capture and definition has also now been devolved to client facing SECs

  System integration, configuration management, VV&T and deployment planning roles remain centralised.

5.4 Acceptance of some duplication

As stated above, CHIS cannot dictate design or time-scales to its client projects or impose change on them which

has no specific business advantage to the client business function. Current client applications are major projects in their own right with large budgets and constrained time-scales. In some cases further issues arise due to the involvement of external contractors.

Even when interface definitions can be maintained as compatible with previous versions, the overheads of regression testing may be unacceptable to clients already deployed with an existing version. CHIS therefore is constrained to support up to three versions of an interface and the underlying software component at any point in time.

CHIS may also deliberately develop multiple RPCs supporting the same functionality. For example, most of the functionality supported by the original three ACE order-entry RPCs will be superseded by a more comprehensive set of 13 currently under development for the VBS telesales client. The existing ACE service is, in parallel, being enhanced to support additional products by a separate development team.

Travelling the evolutionary path towards a single optimised set of RPCs seems likely to be a continuous process of convergence and divergence. It is hoped that as the portfolio of CHIS services is extended this effect will diminish as CHIS services are taken up by a new generation of lightweight, tactical clients.

6. Current status and future

6.1 Live deployment

There are currently five CHIS services deployed on four single-processor Unix servers (HP E55). There are two DCE server types each with a single application interface. They are in use by two ISAP platforms with a maximum throughput of approximately 50 000 RPCs per day. Throughput on a single CHIS machine has been tested up to a minimum of 120 simultaneous RPCs. It is believed that this figure could be substantially higher.

An additional two machines will be deployed early in 1997 to support special services fault tracking and related services for MCSS ServiceView and a further two machines to support enhanced order-entry services for VBS telesales and ServiceView.

6.2 Services under development

Approximately 22 RPCs are currently under development by CHIS covering:

- PSTN order entry, including new customers, accounts and installations, customer appointment making and directory entries,
- customer contact handling for complaints and enquiries,
- PSTN and special services order progression.

In addition, CHIS infrastructure has been made available to the SMART project which is developing CHIS-'branded' services for PSTN fault handling.

6.3 Outlook

CHIS is currently viewed by OSS management as a successful prototype. Its general perspective on technical strategy for the OSS is accepted, though not perhaps always fully understood.

The technical problems have been less than are commonly experienced with the adoption of novel technology on this scale.

The most significant issues which remain are concerned with organisational matters and achieving a balance between short-term speed to market and quality of engineering.

CHIS has not as yet achieved a critical mass in terms of numbers of services. The author doubts whether building up a catalogue of services on the back of a bespoke client-by-client development approach will achieve this quickly enough to allow CHIS to meet its full objectives. To overcome this, CHIS intends to follow a multi-track approach as described in the project model.

Acknowledgements

The author wishes to acknowledge the influence of past and present BT people who have worked on the CHIS project.

References

1 Furley N: 'The BT operational support systems architecture', BT Technol J, 15, No 1, pp ..... (January 1997).

2 Chandler J: 'Management of special services — designing for a changing world', BT Technol J, 15, No 1, pp ..... (January 1997).

3 White P: ‘A service system for BT’s major customers’, BT Technol J, 15, No 1, pp 81—86 (January 1997).

4 Selley C J: ‘SMART — improving customer service’, BT Technol J, 15, No 1, pp 69—80 (January 1997).

5 Chandler J: ‘Management of special services — designing for a changing world’, BT Technol J, 15, No 1, pp 87—97(January 1997).

6 Booch G: ‘Object oriented analysis and design with applications’, 2nd Edition, Benjamin/Cummings Publishing Co (1994).

7 Winton N: ‘Management for Distributed Computing Environment-based applications’, BT Technol J, 15, No 1, pp 160—166 (January 1997).

8 Sims O: ‘Business objects’, McGraw-Hill (1994).

9 Yourdon E: ‘Managing the systems lifecycle’, Prentice Hall (1988).

Graeme Mathieson graduated from the University of Aberdeen in 1973 with a degree in English Language and Linguistics. He joined BT in the North of Scotland and was involved with the Highlands and Islands Development Scheme which replaced the last manual exchanges with automatic switches and completed the introduction of STD. He entered the computing profession with National Girobank in 1977, and while there was among the pioneer users of Data Analysis techniques, ICL’s VME operating system and the experimental packet switchstream service (EPSS). Returning to BT in 1981, he worked as a consultant on data analysis, data standards and data dictionaries, on an evaluation of the Cincinnati Bell Customer Record Information System and on the early analysis phase of the CSS project. He moved to a CSS data conversion and database load role and led the ‘Seamless CSS’ study which triggered the development of CSS transaction switching. Following completion of CSS roll-out, he had responsibility for support and development of CSS Customer Records and Directory sub-systems and various other CSS initiatives. In recent years his work has included consultancy on BTNA systems, pilot client/server developments and various architectural activities including UCH and CHIS.